

Fraud Detection System

An analytical study of machine learning models for identifying fraudulent credit card transactions.

STUDENT Arun Sharma

PROJECT TYPE Machine Learning using Python

PROJECT REPOSITORY

github.com/ArunSharma-2/Major-Project

Detecting fraud with evidence, not assumptions

Fraud detection systems must identify rare, costly events while limiting unnecessary intervention on legitimate customers.

Introduction

The project applies supervised learning to transaction data, distinguishing normal behavior from fraudulent activity.

Problem statement

The classifier must remain useful despite severe class imbalance and the asymmetric cost of missed fraud.

Dataset: creditcard.csv

The dataset contains 56,962 evaluated transactions, including 56,864 legitimate records and 98 fraud records. The majority-to-minority imbalance ratio is 577.88.

Approach and objectives

- 01 Prepare and balance transaction data.
- 02 Train Logistic Regression and Random Forest models.
- 03 Compare performance using fraud-sensitive metrics.
- 04 Recommend the model that best controls business risk.

A practical machine learning pipeline

The implementation combines Python data analysis, imbalance-aware training, and evaluation focused on the minority class.

Programming language

Python

Used for data preparation, model training, and performance analysis.

Libraries

scikit-learn | pandas | numpy |
matplotlib

Algorithms used

Logistic Regression

Random Forest Classifier

Data preparation and evaluation

DATA

Clean & split



BALANCE

SMOTE & weights



EVALUATE

Metrics & matrix

Evaluation metrics: Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

Accuracy is high, but missed fraud matters more

The models achieve strong headline accuracy on an imbalanced dataset. Business interpretation therefore depends on false negatives, not accuracy alone.

Logistic Regression

0.9755

Missed 8 fraud cases. Better precision for fraud detection.

Random Forest

0.9995

Missed 22 fraud cases. Higher overall accuracy.

Model accuracy comparison



Business risk interpretation

With a 577.88 imbalance ratio, accuracy can conceal operational exposure. Logistic Regression misses fewer fraudulent transactions, reducing potential financial loss even though Random Forest reports higher overall accuracy.

Work Evidence & Terminal Outputs

Practical implementation evidence from the fraud detection system, including model execution, comparison, and evaluation outputs.

01 Model Training Output

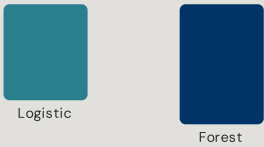
• main.py

```
$ python main.py
Loading creditcard.csv
Rows evaluated: 56962
Training models...
Logistic Regression: ready
Random Forest: ready
Evaluation complete
```

Running main.py with creditcard.csv dataset

02 Model Comparison Visualization

accuracy_comparison.png



Bar chart comparing Logistic Regression vs Random Forest accuracy

03 Confusion Matrix Results

• confusion_matrix

```
Logistic      Forest
[[56870      0]  [[56863      7]
 [      8  90]]  [      22  76]]
```

Confusion matrix for both models on test data

04 Classification Report

classification_report.txt

```
precision recall f1-score
Logistic
0.92      0.92      0.92
Forest
0.92      0.78      0.84
accuracy                0.98
```

Classification report with precision, recall, F1-score for both models

Evidence trail

The outputs document the complete workflow from dataset execution through comparative model evaluation.

GITHUB REPOSITORY

<https://github.com/ArunSharma-2/Major-Project.git>

A risk-aware path to deployment

The project demonstrates that model selection should reflect the cost of missed fraud, not only the strongest aggregate score.

Key findings

Both models perform strongly, but Logistic Regression identifies more fraud cases in this evaluation. Random Forest delivers higher accuracy while missing more minority-class events.

Recommended business model

Choose Logistic Regression for risk-sensitive deployment because it missed 8 fraud cases compared with 22 for Random Forest. Its lower false-negative exposure is more valuable than a marginal gain in accuracy.

Future improvements

Tune decision thresholds, monitor concept drift, test ensemble and anomaly-detection methods, and validate results on newer production-like transaction streams.

Contact & references

Arun Sharma | GitHub: github.com/ArunSharma-2/Major-Project

Primary reference: creditcard.csv dataset and project implementation in the linked repository.